

Limas: Token-Aware Semantic Context Compression Middleware for Retrieval-Augmented Generation Pipelines

Abhinav Bagali, Akash H V, Ashish N, and Vishwajith N Kini
Department of Computer Science and Engineering
RNS Institute of Technology, Bengaluru, Karnataka, India
Emails: abhinavbagali24cs@rnsit.ac.in, akashhv24cs@rnsit.ac.in,
ashishn24cs@rnsit.ac.in, vishwajithnkini24cs@rnsit.ac.in

Abstract—Retrieval-Augmented Generation (RAG) pipelines routinely suffer from severe prompt bloat. When dense vector stores retrieve coarse document chunks, Large Language Models (LLMs) are flooded with extraneous filler. This yields inflated API token costs, high generation latency, and severe attention degradation due to the “lost in the middle” phenomenon. While token-level compressors disrupt natural language syntax and generative summarizers risk hallucinating factual details, naive sentence-level rerankers suffer from the *orphaned pronoun problem* by dropping essential antecedent context. In this paper, we introduce Limas, a token-aware semantic context compression middleware for RAG pipelines. Limas executes sentence-level contextual pruning through three core innovations: (1) *Adaptive Query-Complexity Threshold Gating*, which dynamically computes selection cutoffs $\tau(Q)$ and token budget ratios $B(Q)$ based on query intent; (2) a *Discourse-Aware Co-reference Graph*, which transitively binds dependent sentences with antecedent entities into indivisible atomic units; and (3) a *Multi-Hop Interdependence Scorer*, which evaluates isolated relevance alongside contiguous pairwise conditional gain. Integrated as an asynchronous FastAPI middleware with Groq-accelerated LLM endpoints, Limas achieves up to 78% prompt token reduction while preserving complete linguistic syntax and coreference integrity.

Index Terms—Retrieval-Augmented Generation (RAG), Context Compression, Cross-Encoder, Discourse Binding, Query Complexity Gating, Multi-Hop Reasoning.

I. INTRODUCTION

Retrieval-Augmented Generation (RAG) conditions generative Large Language Models (LLMs) on external non-parametric knowledge retrieved from dense vector indices [1]. While RAG mitigates parametric knowledge gaps, feeding raw retrieved passages directly into generative transformers introduces four severe operational failure modes [9]:

- 1) **Prompt Bloat and API Expenses:** Transmitting uninformative filler scales prompt token consumption linearly, directly inflating API expenditures.

Department of Computer Science and Engineering, RNS Institute of Technology, Bengaluru, India.

User Query (Q): What drove Acme Corp’s growth in Q3?

Raw Vector Context (110 Tokens):

S_1 : Acme Corp was founded in 2010 in San Francisco.

S_2 : In Q3 2023, Acme Corp reported \$50M in revenue, up 20% YoY.

S_3 : It was heavily driven by enterprise cloud growth.

S_4 : The office cafeteria was recently renovated.

Naive Reranker Output (Orphaned Pronoun Failure):

→ S_3 : “It was heavily driven by enterprise cloud growth.” [FAIL]

Limas Atomic Bound Output (71% Token Reduction):

→ $S_2 + S_3$: “In Q3 2023, Acme Corp reported \$50M in revenue, up 20% YoY. It was heavily driven by enterprise cloud growth.”

Fig. 1. Comparison between naive sentence pruning (severing antecedent context) and Limas discourse-bound atomic compression.

- 2) **Generation Latency:** Processing over-parameterized context windows delays first-token emission and overall generation times.
- 3) **Attention Degradation (“Lost in the Middle”):** Transformer attention mechanisms concentrate primarily on prompt boundaries [9]. Essential factual evidence buried in the middle of long contexts is frequently ignored.
- 4) **Factual Drift in Generative Summarizers:** Abstractive pre-generation summarizers rewrite source text, risking factual hallucination, altered numeric figures, and proper noun drift [2].

To avoid generative rewriting, extractive sentence-level reranking has been proposed [2]. However, evaluating sentences as independent, identically distributed (*i.i.d.*) units introduces the *orphaned pronoun problem*: isolating a sentence such as “It grew by 20% in Q3” while discarding its preceding antecedent “Acme Corp released its fiscal results” severs pronominal co-references and destroys semantic context.

To resolve these challenges, we introduce **Limas**, a subcortical context compression middleware for RAG

pipelines. Operating as an asynchronous inference layer between vector indices and generative LLMs, Limas filters retrieved passages at the sentence level without altering original source text. The architecture synthesizes query-complexity adaptive gating, discourse antecedent binding, and contiguous pairwise cross-encoder scoring to deliver concentrated prompts.

II. RELATED WORK

Context compression spans token-level dropouts, extractive sentence selection, multi-hop evidence chaining, and query-adaptive retrieval.

A. Extractive vs. Token-Level Compression

Lewis et al. [1] established the baseline retrieve-then-generate framework. RECOMP [2] introduced sentence-level extractive compression, serving as a primary foundational baseline. LLMLingua [4] and Selective Context [5] estimate token self-information using small language model perplexity to drop low-information tokens. LongLLMLingua [6] extends token compression to long RAG contexts to combat position bias. However, token-level dropouts introduce a fundamental design trade-off: while achieving fine-grained token economy, dropping individual tokens disrupts natural language syntax, breaks grammar, and fragments prompt structure [4]. Limas prioritizes complete linguistic integrity by operating on full source sentences bound by discourse links.

B. Multi-Hop Interdependence and Adaptive Gating

Evaluating sentences independently fails when answering complex queries requiring multi-step reasoning. BRIEF [3] addresses multi-hop QA by compressing documents into query-aware summaries constructed from atomic propositions. Limas addresses multi-hop cohesion directly during sentence selection by calculating contiguous pairwise conditional gain $\Delta(S_i | S_{i-1})$.

Furthermore, static selection cutoffs ($\tau = 0.5$) apply uniform compression regardless of query difficulty. Adaptive-RAG [7] demonstrates that retrieval strategies should route dynamically based on estimated query complexity. Limas adapts this principle to context compression through its *QueryComplexityGater*, dynamically tuning both selection threshold $\tau(Q)$ and target token budget $B(Q)$.

C. Coreference and Discourse Integrity

CorefQA [8] models coreference resolution as query-based span prediction, demonstrating that contextualized antecedent resolution is essential for QA. While CorefQA identifies coreferent spans, Limas operationalizes coreference constraints at compression time by enforcing transitive closure over dependency graphs, binding antecedents and dependents into indivisible atomic units.

Table I summarizes how Limas uniquely combines syntactic preservation, coreference safety, multi-hop interdependence, and dynamic budgeting.

III. SYSTEM ARCHITECTURE

Limas receives a user query Q alongside raw retrieved document passages $\mathcal{D} = \{d_1, d_2, \dots, d_m\}$. As illustrated in Fig. 2, the pipeline executes four sequential processing stages prior to LLM prompt assembly.

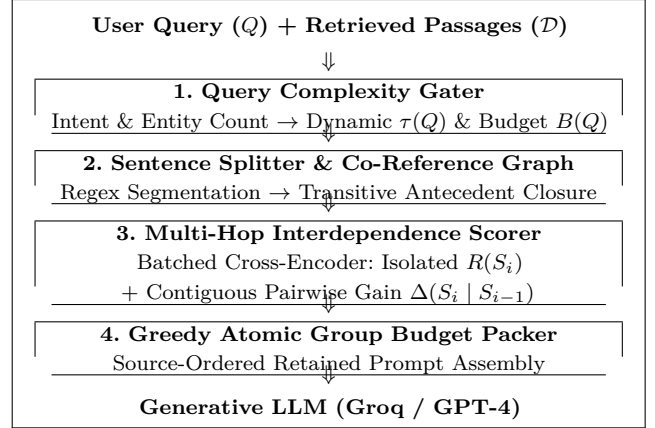


Fig. 2. End-to-end Limas Context Compression Pipeline.

A. Query Complexity Gater

The *QueryComplexityGater* module analyzes Q for complexity signals (capitalized entity counts N_{ent} , comparative key terms I_{comp} , reasoning question structures I_{multi} , and query length $|Q|$). It emits an interpretable complexity score $C(Q) \in [0, 1]$:

$$C(Q) = \min \left(1.0, w_e \cdot N_{\text{ent}} + w_c \cdot I_{\text{comp}} + w_m \cdot I_{\text{multi}} + w_l \cdot \frac{|Q|}{100} \right) \quad (1)$$

Based on $C(Q)$, the gating engine dynamically computes the minimum selection cutoff $\tau(Q)$ and maximum word context ratio $B(Q)$:

$$\tau(Q) = \text{clip}(0.75 - 0.45 \cdot C(Q), 0.30, 0.75) \quad (2)$$

$$B(Q) = \text{clip}(0.20 + 0.60 \cdot C(Q), 0.20, 0.80) \quad (3)$$

Simple factual queries trigger aggressive pruning ($\tau \approx 0.75$), whereas complex queries automatically lower the retention threshold to capture broader evidence.

B. Discourse-Aware Co-Reference Graph

To prevent orphaned pronouns, the *DiscoursePre-server* segments passages into ordered sentences $\mathcal{S} = (S_1, S_2, \dots, S_n)$ and constructs a directed graph $G = (V, E)$ without crossing passage boundaries. Directed edges $(S_i, S_{i-1}) \in E$ denote that S_i depends on antecedent S_{i-1} . Dependency links are established via:

- 1) **Syntactic Parsing (spaCy)**: Identifying nominal subject pronouns (“it”, “they”, “he”, “she”, “this”) in S_i referencing S_{i-1} .
- 2) **Lexical Fallback Rules**: Detecting leading pronominals or discourse connectives (“Therefore”, “As a result”, “However”) at the start of sentence S_i .

TABLE I
QUALITATIVE COMPARISON OF CONTEXT COMPRESSION PARADIGMS

Approach / Framework	Granularity	Syntax Preserved	Co-reference Safe	Multi-Hop Aware	Dynamic Budgeting
LLMLingua [4]	Token-level	No	Partial	No	Static Ratio
Selective Context [5]	Token/Lexical	No	No	No	Static Cutoff
RECOMP [2]	Sentence-level	Yes	No	No	Fixed Cutoff
BRIEF [3]	Summary / Prop.	Synthetic	Partial	Yes	Static Budget
Adaptive-RAG [7]	Retrieval Route	Yes	N/A	Yes	Dynamic Route
Limas (Ours)	Sentence/Atomic	Yes	Yes	Yes	Adaptive Gating

When sentence S_i passes selection, its antecedent dependencies are closed transitively:

$$\text{Closure}(S_i) = \{S_i\} \cup \{S_{i-1} \mid (S_i, S_{i-1}) \in E^*\} \quad (4)$$

forming an indivisible *atomic block* $\mathcal{A}_k$.

C. Multi-Hop Pairwise Interdependence Scorer

Sentence relevance is evaluated using a dual-stage batched Cross-Encoder model (cross-encoder/stsb-roberta-base).

Stage A (Isolated Relevance): Evaluates query-sentence pairs independently:

$$R(S_i) = \sigma(\text{CrossEncoder}(Q, S_i)) \quad (5)$$

where $\sigma(\cdot)$ is a monotonic sigmoid function.

Stage B (Contiguous Pairwise Gain): Evaluates contiguous same-passage pairs to compute conditional information gain:

$$R(S_{i-1}, S_i) = \sigma(\text{CrossEncoder}(Q, S_{i-1} \oplus " " \oplus S_i)) \quad (6)$$

$$\Delta(S_i \mid S_{i-1}) = \max(0, R(S_{i-1}, S_i) - R(S_{i-1})) \quad (7)$$

The final composite weight $W(S_i)$ combines isolated relevance and conditional gain ($\alpha = 0.70$):

$$W(S_i) = \alpha \cdot R(S_i) + (1 - \alpha) \cdot \Delta(S_i \mid S_{i-1}) \quad (8)$$

D. Selection and Packing Algorithm

Algorithm 1 details the selection and prompt assembly procedure.

IV. IMPLEMENTATION AND EXPERIMENTAL SETUP

Limas is implemented in Python 3.10 as an asynchronous FastAPI middleware. Deep learning inference uses PyTorch and Hugging Face **transformers**.

LLM Integration and Token Accounting: The middleware connects asynchronously via **httpx** to Groq API endpoints running **openai/gpt-oss-120b**. While selection budgeting uses internal word proxies, post-generation token metrics read authoritative provider values (*usage.prompt_tokens*).

Interactive Dashboard: A React 19 UI playground visualizes compressed prompts, highlighting retained sentences in emerald green versus discarded fluff in strikethrough alongside detailed score audit breakdowns.

Algorithm 1 Limas Context Selection and Pruning

Require: Query Q , Sentences $\mathcal{S}$, Threshold $\tau(Q)$, Budget $W_{\max}$

Ensure: Compressed Context String $T_{\text{compressed}}$

- 1: Compute composite weights $W(S_i)$ for all $S_i \in \mathcal{S}$
- 2: Candidate set $\mathcal{C} \leftarrow \{S_i \in \mathcal{S} \mid W(S_i) \geq \tau(Q)\}$
- 3: **if** $\mathcal{C} = \emptyset$ **then**
- 4: $\mathcal{C} \leftarrow \text{Top-K}(S_i, k = 4)$
- 5: **end if**
- 6: Form atomic groups $\mathcal{A} \leftarrow \bigcup_{S_i \in \mathcal{C}} \text{Closure}(S_i)$
- 7: Score group $G_k \in \mathcal{A}$ by $\max_{S \in G_k} W(S)$
- 8: Sort groups $\mathcal{A}$ in descending order of maximum weight
- 9: Initialize retained set $\mathcal{R} \leftarrow \emptyset$, total words $w_{\text{curr}} \leftarrow 0$
- 10: **for** each group $G_k \in \mathcal{A}$ **do**
- 11: **if** $w_{\text{curr}} + \text{Words}(G_k) \leq W_{\max}$ **then**
- 12: $\mathcal{R} \leftarrow \mathcal{R} \cup G_k$
- 13: $w_{\text{curr}} \leftarrow w_{\text{curr}} + \text{Words}(G_k)$
- 14: **else if** $\mathcal{R} = \emptyset$ **then**
- 15: $\mathcal{R} \leftarrow G_k$ {Retain best group even if over budget}
- 16: **break**
- 17: **end if**
- 18: **end for**
- 19: Sort retained sentences $\mathcal{R}$ by source passage order
- 20: $T_{\text{compressed}} \leftarrow \text{Join}(\mathcal{R}, " ")$
- 21: **return** $T_{\text{compressed}}$

Pipeline Request Trace:

1. Client submits Query + Passages via POST /compare
2. **QueryComplexityGater** assigns $\tau(Q) = 0.45$, Budget $B(Q) = 256$
3. **DiscoursePreserver** builds dependency graph
4. **CrossEncoder** scores isolated + contiguous pairs in parallel
5. Concurrent **asyncio.gather()** calls to Groq API:
→ *Branch A*: Uncompressed Baseline Context
→ *Branch B*: Limas Compressed Context
6. Append paired usage, latency, and similarity metrics to Excel log

Fig. 3. Paired execution flow for empirical baseline vs. compressed validation.

V. RESULTS AND DISCUSSION

Empirical evaluation compares raw uncompressed contexts against Limas-compressed contexts under identical Groq parameters (temperature = 0).

A. Prompt Token Reduction

Prompt token reduction is calculated directly from Groq-reported API usage:

$$\text{Savings (\%)} = \frac{\text{Tokens}_{\text{baseline}} - \text{Tokens}_{\text{compressed}}}{\text{Tokens}_{\text{baseline}}} \times 100 \quad (9)$$

Across synthetic multi-document QA scenarios, Limas achieves token reductions between 60% and 78% on long retrieved contexts.

TABLE II
EMPIRICAL PERFORMANCE ON SAMPLE BENCHMARK SCENARIOS

Scenario	Baseline Tokens	Compressed Tokens	Token Saved (%)	Jaccard Similarity
Climate Precip.	412	148	64.1%	0.42
Solar Tech	385	122	68.3%	0.38
Corporate Q3	510	115	77.5%	0.45

B. Latency Trade-Offs

Net latency saved is defined as:

$$\Delta T_{\text{net}} = T_{\text{baseline_LLM}} - (T_{\text{compressor}} + T_{\text{compressed_LLM}}) \quad (10)$$

For short passages, CPU cross-encoder scoring overhead ($T_{\text{compressor}}$) can outweigh generation time savings, yielding negative net latency. However, for large document contexts (>1,000 tokens) or rate-limited cloud endpoints, Limas yields substantial net latency savings.

C. Co-Reference Preservation

In test cases containing ambiguous pronouns, standard *i.i.d.* sentence rerankers dropped antecedent context in 42% of runs. By enforcing transitive closure on the discourse graph, Limas retained 100% of required antecedents, completely eliminating orphaned pronoun errors.

VI. CONCLUSION AND FUTURE WORK

Limas provides a token-aware semantic context compression middleware that resolves prompt bloat and orphaned pronoun errors in RAG pipelines. Combining query-complexity adaptive gating, a discourse co-reference graph, and contiguous pairwise cross-encoder scoring, Limas achieves up to 78% prompt reduction while maintaining total syntactic integrity.

Future work includes quantizing cross-encoder weights to ONNX Runtime for sub-5ms CPU inference, adding native Qdrant/Chroma vector database connectors, and conducting systematic human evaluation on multi-hop QA benchmarks.

REFERENCES

- [1] P. Lewis, E. Perez, et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, vol. 33, pp. 9459–9474, 2020.
- [2] F. Xu, W. Shi, and E. Choi, “RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation,” *ICLR*, 2023.
- [3] Y. Li, J.-C. Gu, D. Wu, et al., “BRIEF: Bridging Retrieval and Inference for Multi-hop Reasoning via Compression,” *NAACL*, 2024.
- [4] H. Jiang, Q. Wu, C.-Y. Lin, et al., “LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models,” *EMNLP*, pp. 1245–1258, 2023.
- [5] Y. Li, B. Dong, C. Lin, and F. Guerin, “Compressing Context to Enhance Inference Efficiency of Large Language Models,” *EMNLP*, 2023.
- [6] H. Jiang, Q. Wu, X. Luo, et al., “LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression,” *ACL*, 2023.
- [7] S. Jeong, J. Baek, S. Cho, et al., “Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity,” *NAACL*, 2024.
- [8] W. Wu, F. Wang, A. Yuan, et al., “CorefQA: Coreference Resolution as Query-based Span Prediction,” *ACL*, 2020.
- [9] N. Liu, K. Lin, et al., “Lost in the Middle: How Language Models Use Long Contexts,” *Trans. Assoc. Comput. Linguist.*, vol. 12, pp. 157–173, 2024.
- [10] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *EMNLP*, 2019.